

C04-AT2-CASE STUDY SCENARIO

Hospital Management System (HMS) – Case Study Solution

Part A

1. Limitations of the Existing System

1. **Redundant Data Storage**
 - Patient information is stored multiple times, leading to duplication and inconsistency.
2. **Rigid Billing System**
 - Different billing methods for inpatients, outpatients, and emergency patients are difficult to manage.
3. **Poor Extensibility**
 - Adding new patient categories requires major code modifications.
4. **Memory Inefficiency**
 - Static memory allocation wastes resources.
5. **Lack of Reusability**
 - Similar code is repeated for different patient types.
6. **Maintenance Difficulty**
 - Updating the system becomes complex and time-consuming.

2. How OOP Principles Solve These Issues

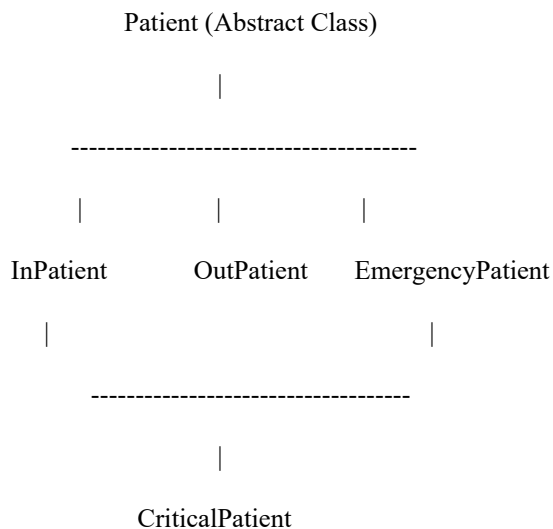
OOP Principle	Solution
Encapsulation	Protects patient data using private members
Inheritance	Reuses common patient properties
Polymorphism	Supports different billing methods
Abstraction	Hides unnecessary implementation details
Composition	Doctor object inside Patient class
Dynamic Binding	Executes correct billing function at runtime

3. Functional Requirements

- Add patient records
- Display patient details
- Calculate bills
- Manage inpatient details
- Manage outpatient details
- Manage emergency cases
- Assign doctors
- Handle critical patients
- Dynamic memory management
- Generate reports

Part B

1. Class Hierarchy



2. Multiple Inheritance

```
class CriticalPatient : public InPatient, public EmergencyPatient
{
};
```

3. Virtual Base Class

To avoid duplicate copies of Patient in the diamond structure:

```
class InPatient : virtual public Patient
```

```
{  
};
```

```
class EmergencyPatient : virtual public Patient
```

```
{  
};
```

```
class CriticalPatient : public InPatient, public EmergencyPatient
```

```
{  
};
```

4. Composition

```
class Doctor
```

```
{  
    string doctorName;  
    string specialization;  
};
```

```
class Patient
```

```
{  
    Doctor doc; // Composition  
};
```

Part C

1. Inheritance vs Composition

Inheritance	Composition
IS-A relationship	HAS-A relationship
Patient IS-A person	Patient HAS-A Doctor
Reuse behavior	Reuse objects
Tight coupling	Loose coupling

2. Static vs Dynamic Binding

Static Binding	Dynamic Binding
Compile-time	Run-time
Function Overloading	Virtual Functions
Faster	Flexible

Example:

```
Patient *p;
```

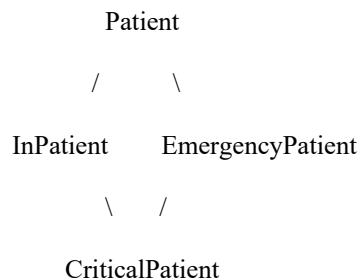
```
p = new InPatient();
```

```
p->calculateBill();
```

3. Role of Virtual Base Classes

Problem:

Without virtual inheritance:



CriticalPatient gets two copies of Patient.

Solution:

virtual public Patient

Only one copy of Patient exists.

4. Benefits of Pointers

- Dynamic memory allocation
- Runtime polymorphism
- Efficient memory usage
- Easy handling of multiple patients

Example:

```
Patient *p = new InPatient();
```

5. Design Justification

- Reusability through inheritance
- Flexibility through polymorphism
- Better memory management
- Easy maintenance
- Scalability for future patient types

Part D – C++ Program

```
#include <iostream>

using namespace std;

// Doctor Class (Composition)

class Doctor
{
public:
    string name;

    string specialization;

    Doctor(string n = "Unknown", string s = "General")
    {
        name = n;

        specialization = s;
    }

    void displayDoctor()
    {
        cout << "Doctor: " << name << endl;

        cout << "Specialization: " << specialization << endl;
    }
};

// Abstract Class

class Patient
{
protected:
    int patientID;

    string patientName;

    Doctor doc;
```

public:

Patient(int id, string name, Doctor d)

{

 this->patientID = id;

 this->patientName = name;

 this->doc = d;

}

virtual float calculateBill() = 0;

virtual void display()

{

 cout << "\nPatient ID: " << patientID;

 cout << "\nPatient Name: " << patientName << endl;

 doc.displayDoctor();

}

virtual ~Patient() {}

};

// Virtual Inheritance

class InPatient : virtual public Patient

{

protected:

 float roomCharge, treatmentCost;

public:

 InPatient(int id, string name, Doctor d,

 float room, float treat)

 : Patient(id, name, d)

{

 roomCharge = room;

 treatmentCost = treat;

```

    }

    float calculateBill()

    {

        return roomCharge + treatmentCost;

    }

};

class OutPatient : virtual public Patient

{

protected:

    float consultationFee;

public:

    OutPatient(int id, string name, Doctor d,

                float fee)

        : Patient(id, name, d)

    {

        consultationFee = fee;

    }

    float calculateBill()

    {

        return consultationFee;

    }

};

class EmergencyPatient : virtual public Patient

{

protected:

    float emergencyCharge;

public:

    EmergencyPatient(int id, string name, Doctor d,

```



```

        float charge)
    : Patient(id, name, d)
    {
        emergencyCharge = charge;
    }

    float calculateBill()
    {
        return emergencyCharge;
    }
};

// Multiple Inheritance

class CriticalPatient : public InPatient,
                       public EmergencyPatient
{
public:
    CriticalPatient(int id, string name, Doctor d,
                   float room, float treat,
                   float emergency)
        : Patient(id, name, d),
          InPatient(id, name, d, room, treat),
          EmergencyPatient(id, name, d, emergency)
    {
    }

    float calculateBill()
    {
        return roomCharge +
               treatmentCost +
               emergencyCharge;
    }
};

```

```

    }
};

int main()
{
    int choice;

    cout << "1. InPatient\n";
    cout << "2. OutPatient\n";
    cout << "3. EmergencyPatient\n";
    cout << "4. CriticalPatient\n";
    cout << "Enter choice: ";

    cin >> choice;

    Patient *p;
    Doctor d("Dr. Kumar", "Cardiology");

    if(choice == 1)
    {
        p = new InPatient(101, "Ravi", d,
                        2000, 3000);
    }

    else if(choice == 2)
    {
        p = new OutPatient(102, "Arun", d,
                        1000);
    }

    else if(choice == 3)
    {
        p = new EmergencyPatient(103, "Vijay",
                        d, 1500);
    }
}

```

```
else
{
    p = new CriticalPatient(104, "Ajay",
                           d, 2000,
                           3000, 1500);
}
p->display();

cout << "\nTotal Bill = Rs."
      << p->calculateBill();

delete p;

return 0;
}
```

Sample Output

Input:

Choice: 4

Room Charge = 2000

Treatment Cost = 3000

Emergency Charge = 1500

Output:

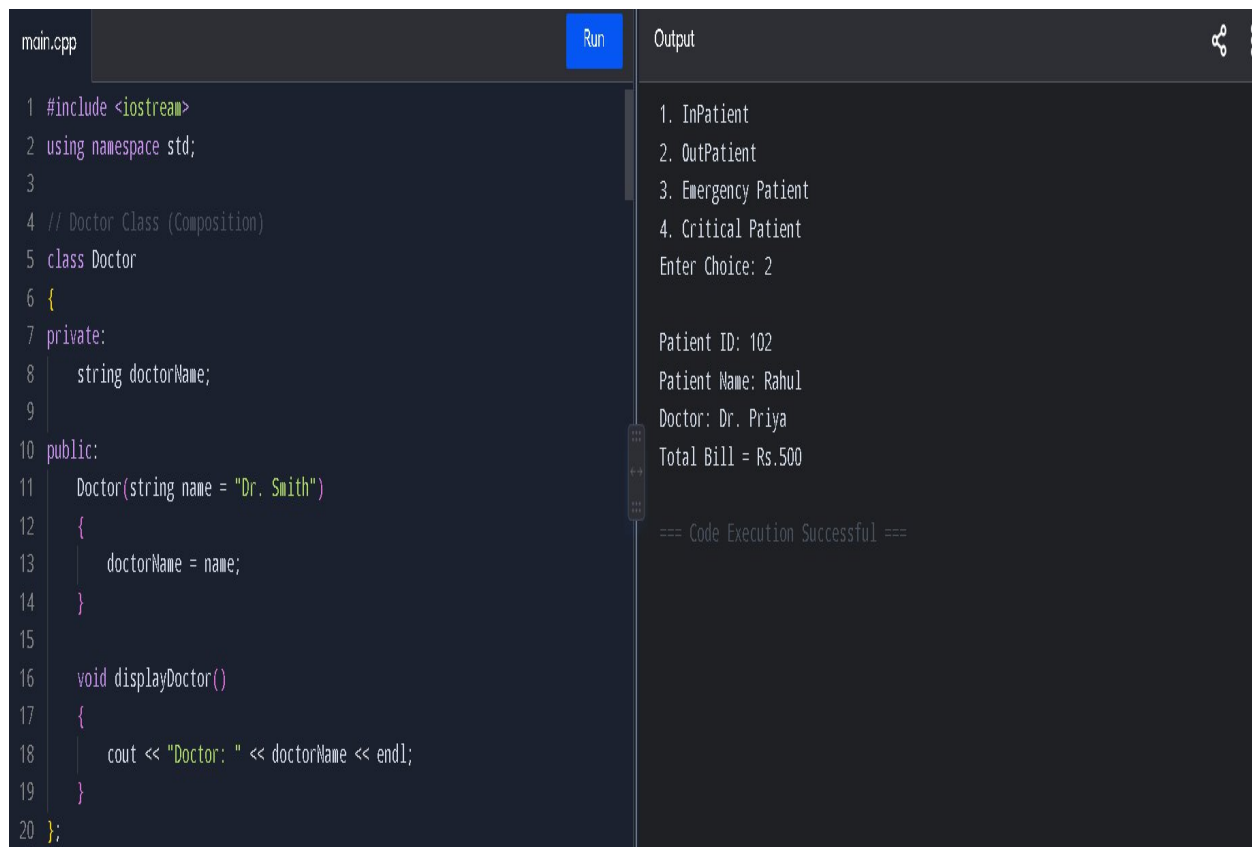
Patient ID: 104

Patient Name: Ajay

Doctor: Dr. Kumar

Specialization: Cardiology

Total Bill = Rs.6500



The image shows a C++ IDE with a file named `main.cpp`. The code defines a `Doctor` class with a private member `doctorName` and a public constructor and display method. The output window shows the execution results, including a menu of patient types, a choice of 2, and the resulting patient details and bill amount.

```
1 #include <iostream>
2 using namespace std;
3
4 // Doctor Class (Composition)
5 class Doctor
6 {
7 private:
8     string doctorName;
9
10 public:
11     Doctor(string name = "Dr. Smith")
12     {
13         doctorName = name;
14     }
15
16     void displayDoctor()
17     {
18         cout << "Doctor: " << doctorName << endl;
19     }
20 };
```

Output

```
1. InPatient
2. OutPatient
3. Emergency Patient
4. Critical Patient
Enter Choice: 2

Patient ID: 102
Patient Name: Rahul
Doctor: Dr. Priya
Total Bill = Rs.500

=== Code Execution Successful ===
```

Conclusion

The redesigned Hospital Management System uses advanced OOP concepts such as abstraction, inheritance, virtual base classes, composition, pointers, and runtime polymorphism. This approach removes redundancy, improves memory management, enables flexible billing, and makes the system scalable and maintainable for future patient categories.